



NutriBot

Project Knowledge Base

Production-ready AI nutrition assistant with deterministic calculations, LangGraph orchestration, RAG knowledge injection, CAG runtime context, and strict validation before returning structured meal plans.

FastAPI	LangGraph	LangChain	OpenAI	FAISS	PostgreSQL	Next.js
---------	-----------	-----------	--------	-------	------------	---------

Version	0.1.0
Python	≥ 3.12
Primary LLM	gpt-4o-mini (configurable)
Default Port	8000 (FastAPI backend)
Food DB	data/food_db.json (single source of truth)
Vector Store	FAISS (rag/faiss_index)

1 · Project Overview

NutriBot is a production-ready AI nutrition assistant. It accepts a user health profile and a natural-language query and returns a structured, validated daily meal plan. The system is intentionally split into a **deterministic computation layer** (BMR, TDEE, macro targets, and food macro values) and an **LLM generation layer** that is strictly constrained by the deterministic values. The LLM may *never* invent macros; every number in the final plan is cross-checked against the food database before the response is returned.

Design Philosophy

- **Safety first:** All calorie/protein/macro calculations are performed by deterministic Python code, not the LLM.
- **Validation closes the loop:** LangGraph retries invalid generations automatically, up to a configurable maximum.
- **RAG for knowledge, not macros:** PDF-ingested nutrition guidelines enrich the LLM context but the food DB is the only source of macro truth.
- **CAG for personalisation:** A context-augmented generation (CAG) object scopes the LLM to the user's allowed foods, restrictions, and targets at every generation.
- **Structured output:** The API always returns Pydantic-validated JSON.

Repository Layout

Path	Purpose
backend/	FastAPI app, Pydantic schemas, async DB (SQLAlchemy + PostgreSQL/SQLite)
agent/	LangGraph graph, nodes, state type, prompts, and CAG context builder
tools/	Deterministic calculator, food catalog loader/filter, validation engine, LangChain wrappers
rag/	PDF ingestion → FAISS index builder, semantic retriever, fallback corpus
data/food_db.json	Single source of truth for all food macro values
frontend/	Next.js App Router UI with profile controls, macro chart, and meal cards
tests/	pytest test suite
main.py	Uvicorn entry-point (uvicorn backend.main:app)
pyproject.toml	Project metadata and dependencies

2 · System Architecture

A request enters through the **FastAPI backend** and is handed to the **LangGraph agent**. The graph is a directed acyclic state-machine with a conditional retry edge. Each node receives the full `NutriBotState TypedDict` and returns an updated copy.

Request Flow

Step	Node	What happens
1	calculate	BMR → TDEE → target calories & protein (Mifflin-St Jeor + activity multiplier + goal delta). Pure Python, no LLM.
2	retrieve	Semantic search over FAISS vector store (or keyword-scored fallback corpus). Returns 4 nutrition-knowledge chunks tagged with relevance.
3	context	Builds the CAG context: user profile, computed targets, fat % constraint, allowed foods list (up to 100), restrictions, and preferences.
4	generate	Formats MEAL_PLAN_PROMPT with CAG context + RAG docs + query. Calls ChatOpenAI (temperature=0). Parses response into a MealPlan.
5	validate	Runs validate_plan(): calorie window, protein floor, fat%, macro consistency, per-item food DB cross-check, allergen/diet/restriction checks.
6	retry / END	If errors and attempts < max_retries → retry node appends error details to query → back to generate. Otherwise END.

LangGraph State: NutriBotState

All nodes communicate through a shared `TypedDict`. Every field is optional (`total=False`) except `profile` and `query` which are set by the caller.

Field	Type	Set by
profile	UserProfile	Caller (API)
query	str	Caller / retry node
targets	NutritionTargets	calculate node
cag_context	CagContext	context node
allowed_foods	list[dict]	context node
rag_docs	list[str]	retrieve node
raw_generation	str	generate node
plan	MealPlan	generate node
validation_errors	list[str]	validate / generate node
attempts	int	Caller (1), incremented by retry
error	str	generate node (on exception)

3 · Data Models & Schemas

All models are Pydantic v2 BaseModelS defined in backend/schemas.py. StrEnum classes enforce valid string values throughout.

Enumerations

Enum	Values
Gender	male · female
ActivityLevel	sedentary · light · moderate · active · very_active
Goal	fat_loss · lean_loss · maintenance · lean_bulk · bulk
DietType	veg · non_veg · vegan

UserProfile

The central input model. Validated on arrival; lists are normalised to lowercase stripped strings.

Field	Type / Constraint	Notes
age	int ge=13, le=100	
gender	Gender	
weight	float gt=20, lt=300	kg
height	float gt=100, lt=250	cm
activity_level	ActivityLevel	
goal	Goal	
diet_type	DietType	
medical_conditions	list[str]	normalised to lowercase
allergies	list[str]	normalised to lowercase
food_preferences	list[str]	used in food ranking
region	str None	used to boost regional foods

FoodDbItem (data/food_db.json)

Field	Type	Notes
id	str	e.g. FOOD_006
food	str	normalised to lowercase
quantity_grams	float	canonical serving size
calories	float	kcal per serving
protein	float	g per serving
carbs	float	g per serving
fat	float	g per serving
diet_types	list[DietType]	veg / non_veg / vegan

allergens	list[str]	e.g. milk, gluten
regions	list[str]	e.g. indian
meal_types	list[str]	breakfast / lunch / dinner / snack
glycemic_index	str None	very_low / low / medium / high
medical_tags	list[str]	e.g. diabetes_safe, high_sodium
tags	list[str]	e.g. high_protein, dairy

NutritionTargets

Produced by the calculate node. All values are deterministic floats rounded to 2 d.p.

bmr	Basal Metabolic Rate (Mifflin-St Jeor, kcal/day)
tdee	Total Daily Energy Expenditure = BMR × activity factor
target_calories	TDEE + goal adjustment (−400 to +400 kcal)
protein_target	weight_kg × 1.8 g/day

Other Key Models

Model	Description
CagContext	Full CAG payload: profile, computed targets, fat% constraint, restrictions, preferences, allowed_foods list, region.
MealPlan	List[Meal] + DailyTotals. This is the LLM output schema.
Meal	name (str) + items (List[MealItem]) + total_calories (float).
MealItem	food, quantity ("N grams"), calories, protein, carbs, fat.
DailyTotals	calories, protein, carbs, fat aggregated across all meals.
GeneratePlanResponse	status, targets, cag_context, retrieved_knowledge, plan, validation_errors.

4 · Tools Layer (tools/)

The tools layer contains pure Python deterministic logic. None of these functions call the LLM. They are the ground-truth computation boundary of the system.

calculator.py — calculate_targets()

Mifflin-St Jeor formula:

```
BMR = 10×weight + 6.25×height - 5×age + (5 if male else -161)
TDEE = BMR × ACTIVITY_FACTORS[activity_level]
target_calories = TDEE + GOAL_ADJUSTMENTS[goal]
protein_target = weight_kg × 1.8
```

Activity Factors

sedentary	×1.200 (little or no exercise)
light	×1.375 (1-3 days/week)
moderate	×1.550 (3-5 days/week)
active	×1.725 (6-7 days/week)
very_active	×1.900 (hard exercise, physical job)

Goal Calorie Adjustments

fat_loss	−400 kcal
lean_loss	−200 kcal
maintenance	±0 kcal
lean_bulk	+200 kcal
bulk	+400 kcal

food_catalog.py — get_allowed_foods()

Loads data/food_db.json (LRU-cached). Filters by diet type, allergens, and medical safety. Then ranks by region match, preference tag overlap, and protein content. Returns at most 100 foods.

Medical Safety Rules (cond_safe)

- **Diabetes:** Exclude foods with glycemic_index not in {low, very_low} unless tagged diabetes_safe.
- **Hypertension / High blood pressure:** Exclude foods tagged high_sodium or hypertension_avoid.
- **Kidney / CKD:** Exclude foods tagged kidney_avoid. Also exclude foods with protein > 25 g unless tagged controlled_protein.

validator.py — validate_plan()

Runs after every generation and is the final safety gate. Failures trigger a retry.

Check	Rule
Calorie window	$\text{abs}(\text{daily_calories} - \text{target_calories}) \leq 50 \text{ kcal}$
Protein floor	$\text{daily_protein} \geq \text{protein_target}$
Macro consistency	$\text{abs}((P \times 4 + C \times 4 + F \times 9) - \text{daily_calories}) \leq 75 \text{ kcal}$
Fat percentage	$20\% \leq (\text{fat} \times 9 / \text{daily_calories}) \times 100 \leq 35\%$

Valid food	<code>food.lower()</code> must exist in <code>food_db_by_name()</code>
Allowed food	food must appear in <code>get_allowed_foods(profile)</code>
Diet compliance	<code>profile.diet_type</code> $\in$ <code>food.diet_types</code>
Allergen check	<code>profile.allergies</code> $\cap$ <code>food.allergens</code> = $\emptyset$
Quantity format	<code>item.quantity</code> == <code>f'{quantity_grams} grams'</code>
Per-field macro match	$\text{abs}(\text{item.field} - \text{food_db.field}) \leq 0.2$ for cal/prot/carb/fat
Meal total	$\text{abs}(\text{sum}(\text{item.cal}) - \text{meal.total_calories}) \leq 10$ kcal
Daily recomputed	$\text{abs}(\text{recomputed_daily} - \text{declared_daily}) \leq 10$ for each macro

5 · RAG System (rag/)

The RAG system enriches the LLM prompt with relevant nutrition-science text. It uses FAISS for semantic search with metadata filtering. When no FAISS index exists a deterministic keyword-scored fallback corpus is used instead.

Ingestion Pipeline (rag/ingest.py)

- Place PDF files in rag/source_docs/. Supported filenames have predefined metadata mappings.
- Run ingest_documents(). PDFs are loaded via LangChain's PyPDFLoader.
- Text is split with RecursiveCharacterTextSplitter (chunk_size=1800, chunk_overlap=150).
- Each chunk is tagged with metadata: source, type (medical/regional/macro/general), condition.
- OpenAIEmbeddings are computed and a FAISS index is saved to rag/faiss_index/.

Supported PDF Sources

Filename	Type	Condition
diabetes_guidelines.pdf	medical	diabetes
pcos_nutrition.pdf	medical	pcos
thyroid_diet.pdf	medical	thyroid
hypertension_diet.pdf	medical	hypertension
indian_diet_guidelines.pdf	regional	general
protein_requirements.pdf	macro	general

Retrieval (rag/retriever.py)

NutritionRetriever.retrieve(query, profile, k=4) works as follows:

- **Metadata filter:** If the user has a supported medical condition (diabetes, pcos, thyroid, hypertension, kidney) a metadata filter is set to prefer matching document chunks.
- **Semantic search:** Search query is enriched with the user's goal and medical conditions. First k results are fetched with the metadata filter; if fewer than k are found the filter is relaxed.
- **Fallback:** If no FAISS index exists, DEFAULT_DOCUMENTS (rag/knowledge_base.py) are keyword-scored and the top-k non-zero scoring documents are returned.
- **Format:** Each chunk is returned as a string prefixed with [source=... type=... condition=...] for traceability.

6 · Backend API (backend/)

FastAPI application at backend/main.py. The database layer uses SQLAlchemy 2 async engine with PostgreSQL (production) or SQLite +aiosqlite (development default).

API Endpoints

Method	Path	Request Body	Response	Notes
POST	/generate-plan	GeneratePlanRequest	GeneratePlanResponse	Main meal plan generation endpoint. Runs the full LangGraph workflow.
POST	/chat	ChatRequest	GeneratePlanResponse	Alias: wraps /generate-plan with message as query.
GET	/user-profile	?user_id=N (query param)	UserProfileResponse	Fetch saved user profile by ID.
POST	/user-profile	CreateUserProfileRequest	UserProfileResponse	Persist a new user profile to the database.
POST	/save-plan	SavePlanRequest	{status, plan_id}	Save a generated meal plan for a user.
GET	/health	—	{status: ok}	Liveness check; also verifies DB connection.

Database Models (backend/models.py)

User	Stores UserProfile fields (age, gender, weight, height, activity_level, goal, diet_type, medical_conditions, allergies).
Plan	Stores generated plans. Fields: id, user_id (FK), date, calories, protein, carbs, fat, plan_json (JSON column for meal details).

Configuration (backend/config.py + env vars)

Settings are loaded via pydantic-settings from environment variables prefixed NUTRIBOT_ or from a .env file.

Env Var	Default	Description
NUTRIBOT_DATABASE_URL	sqlite+aiosqlite:///./nutribot.db	SQLAlchemy async DB URL
NUTRIBOT_OPENAI_API_KEY	(required for generation)	OpenAI API key
NUTRIBOT_MODEL_NAME	gpt-4o-mini	LangChain ChatOpenAI model
NUTRIBOT_MAX_GENERATION_RETRIES	3	Max LangGraph retry attempts
NUTRIBOT_FOOD_DB_PATH	data/food_db.json	Path to food database JSON
NUTRIBOT_RAG_SOURCE_DIR	rag/source_docs	Directory of source PDFs
NUTRIBOT_VECTOR_STORE_PATH	rag/faiss_index	FAISS index save/load path

7 · Non-Negotiable Safety Rules

These rules are hard-coded into the system and cannot be overridden by prompt engineering or user input.

1. LLM cannot calculate nutrition	tools/calculator.py owns all BMR, TDEE, target calorie, and protein calculations. The LLM receives the pre-computed values and must use them verbatim.
2. LLM cannot invent foods	Every generation prompt includes only foods from get_allowed_foods(profile) via the CAG context. The validate step rejects any food not in the DB.
3. Macro values come from food_db.json	RAG documents are nutrition guidance text only. Per-food macro values are always taken from data/food_db.json, never from the LLM or RAG.
4. Allergen / diet / medical enforcement	get_allowed_foods() filters foods before the LLM sees them. validate_plan() re-checks every item independently, so a compliant-looking generation cannot sneak through allergy or medical violations.
5. Quantity is canonical	The LLM must output item.quantity as exactly '{food.quantity_grams} grams'. Any deviation is caught and the generation is retried.
6. Automatic retry with error feedback	LangGraph automatically retries invalid generations, appending the validation error messages to the query so the LLM can self-correct. After max_retries the plan is returned with validation_errors populated and status='error'.
7. Structured JSON output only	The API always returns Pydantic-validated GeneratePlanResponse. Partial or malformed LLM output is treated as a validation error, not surfaced raw.

8 · Setup & Running

Backend

```
# 1. Install dependencies
pip install -e ".[dev]"

# 2. Set required env var
export NUTRIBOT_OPENAI_API_KEY="sk-..."

# 3. Start the FastAPI server
python main.py

# → uvicorn listening on http://0.0.0.0:8000
```

Frontend

```
cd frontend
npm install
export NEXT_PUBLIC_API_BASE_URL="http://localhost:8000"
npm run dev

# → Next.js dev server on http://localhost:3000
```

Build the RAG Index (optional but recommended)

```
# Place PDFs in rag/source_docs/
# diabetes_guidelines.pdf
# pcos_nutrition.pdf
# thyroid_diet.pdf
# hypertension_diet.pdf
# indian_diet_guidelines.pdf
# protein_requirements.pdf

python -c "
from rag.ingest import ingest_documents
ingest_documents('rag/source_docs', 'rag/faiss_index')
"
```

Run Tests

```
pytest
```

Test Suite

File	What it tests
tests/test_calculator.py	BMR, TDEE, calorie targets, protein targets for all genders, activity levels, goals.
tests/test_context.py	CAG context builder: targets propagation, constraints, allowed_foods list.
tests/test_food_catalog.py	Food DB loading, allergen filtering, diet type filtering, medical condition safety.
tests/test_validator.py	validate_plan: calorie window, protein floor, macro consistency, fat%, invalid food, quantity mismatch.

Optional Environment Variables (Windows PowerShell)

```
$env:NUTRIBOT_DATABASE_URL="postgresql+asyncpg://user:pw@host:5432/nutribot"
$env:NUTRIBOT_MODEL_NAME="gpt-4o"
```

```
$env:NUTRIBOT_MAX_GENERATION_RETRIES="5"  
$env:NUTRIBOT_FOOD_DB_PATH="data/food_db.json"  
$env:NUTRIBOT_RAG_SOURCE_DIR="rag/source_docs"  
$env:NUTRIBOT_VECTOR_STORE_PATH="rag/faiss_index"
```

9 · LLM Prompt Design

The meal plan prompt (agent/prompts.py) is a two-message ChatPromptTemplate. The system message establishes the strict engine role. The human message injects all runtime context.

System Message

You are a strict nutrition planning engine. You only generate meal plans.
You must not perform independent nutrition calculations.
Use only USER CONTEXT allowed_foods values.

Human Message Structure

- **USER CONTEXT** — Full CagContext JSON: user profile, targets, constraints, restrictions, preferences, allowed_foods, region.
- **RETRIEVED KNOWLEDGE** — Up to 4 RAG document chunks (tagged with source/type/condition).
- **CRITICAL RULES** — Explicit enumeration of all constraints (foods, macros, quantities, JSON schema).
- **USER QUERY** — The original user query (possibly augmented with retry error details).

Expected Output Schema

```
{
  "meals": [
    {
      "name": "Breakfast",
      "items": [
        {
          "food": "string",
          "quantity": "100 grams",
          "calories": 0,
          "protein": 0,
          "carbs": 0,
          "fat": 0
        }
      ],
      "total_calories": 0
    }
  ],
  "daily_totals": {
    "calories": 0,
    "protein": 0,
    "carbs": 0,
    "fat": 0
  }
}
```

Retry Behaviour

If validate_plan() returns errors and attempts < max_retries, the retry node appends the validation errors to the query:

```
retry_query = (
  f"{state['query']}\n\n"
  f"Previous output failed validation. Fix these errors without "
  f"changing the deterministic target values: {errors}"
)
```

```
)
```

temperature=0 is set on the ChatOpenAI client to maximise determinism across retry attempts.

Generated automatically from source code · NutriBot v0.1.0 · April 2026